PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

API REST CON EXPRESS, MYSQL

Presentado a:	Yoli Alexandra muñoz
Por Aprendiz:	**Beckan Hungria**
Ficha:	3312932

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día **28** de **octubre** del año **2026**

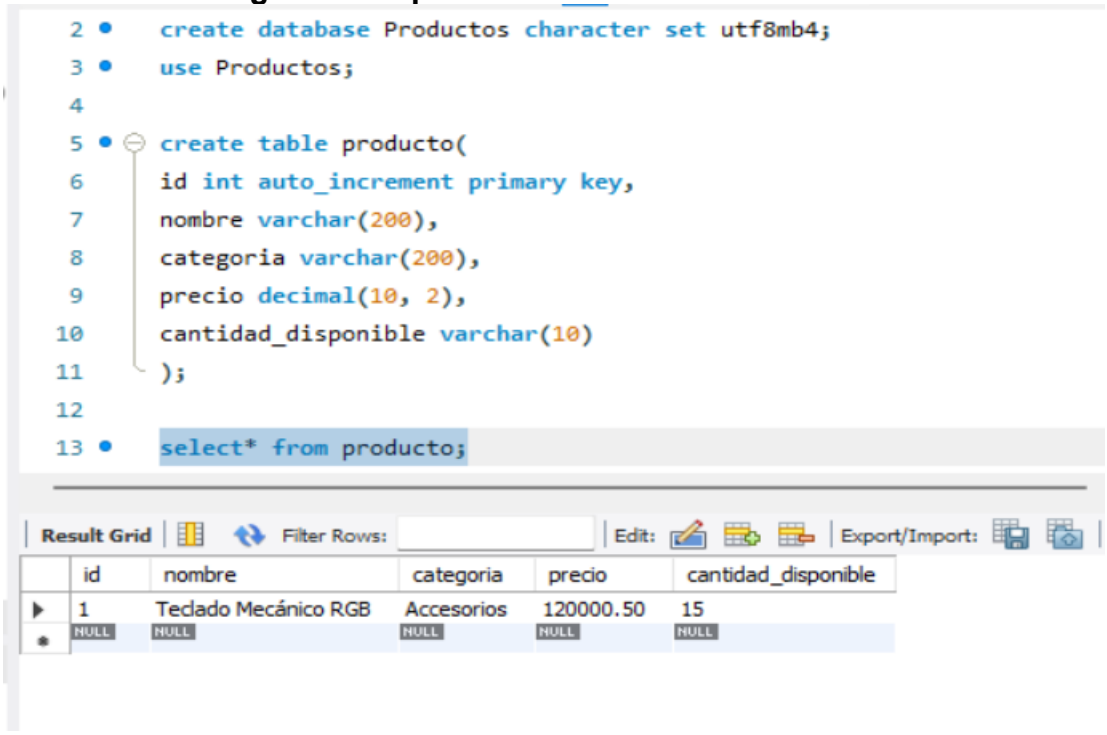
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

1. Actividad o ejercicio 1.

Pantallazo 1: Estructura de la tabla

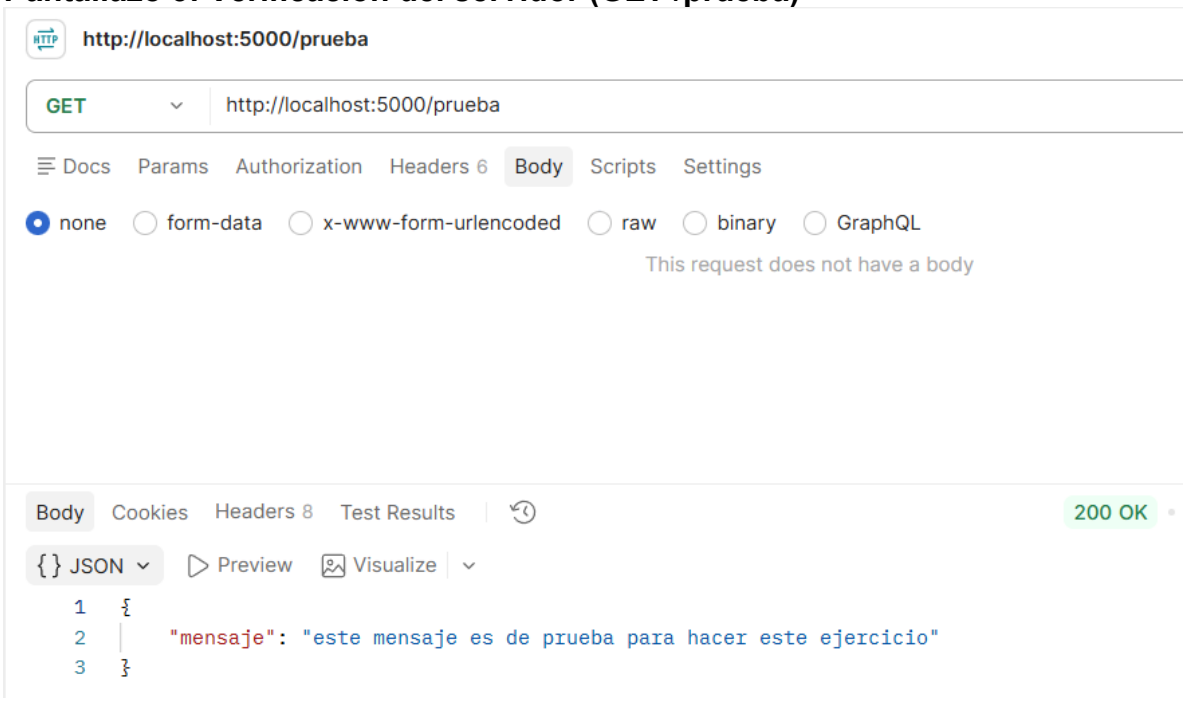


Pantallazo 2: Registros de prueba

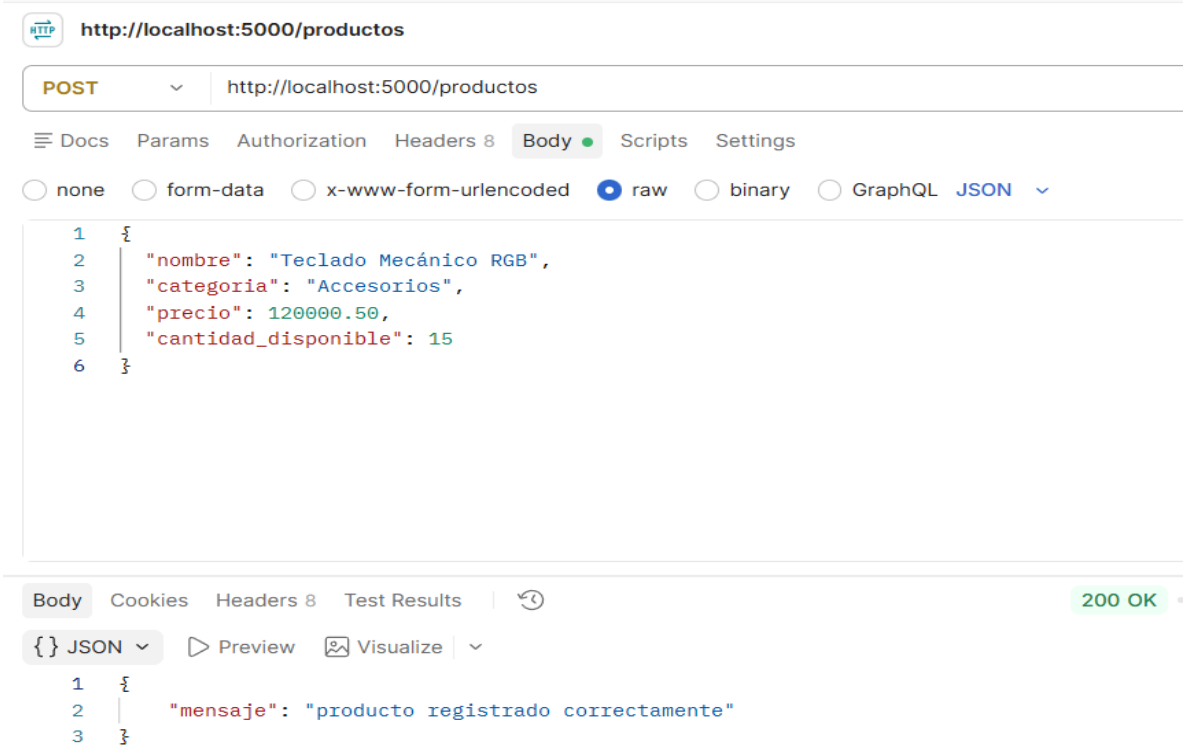


PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

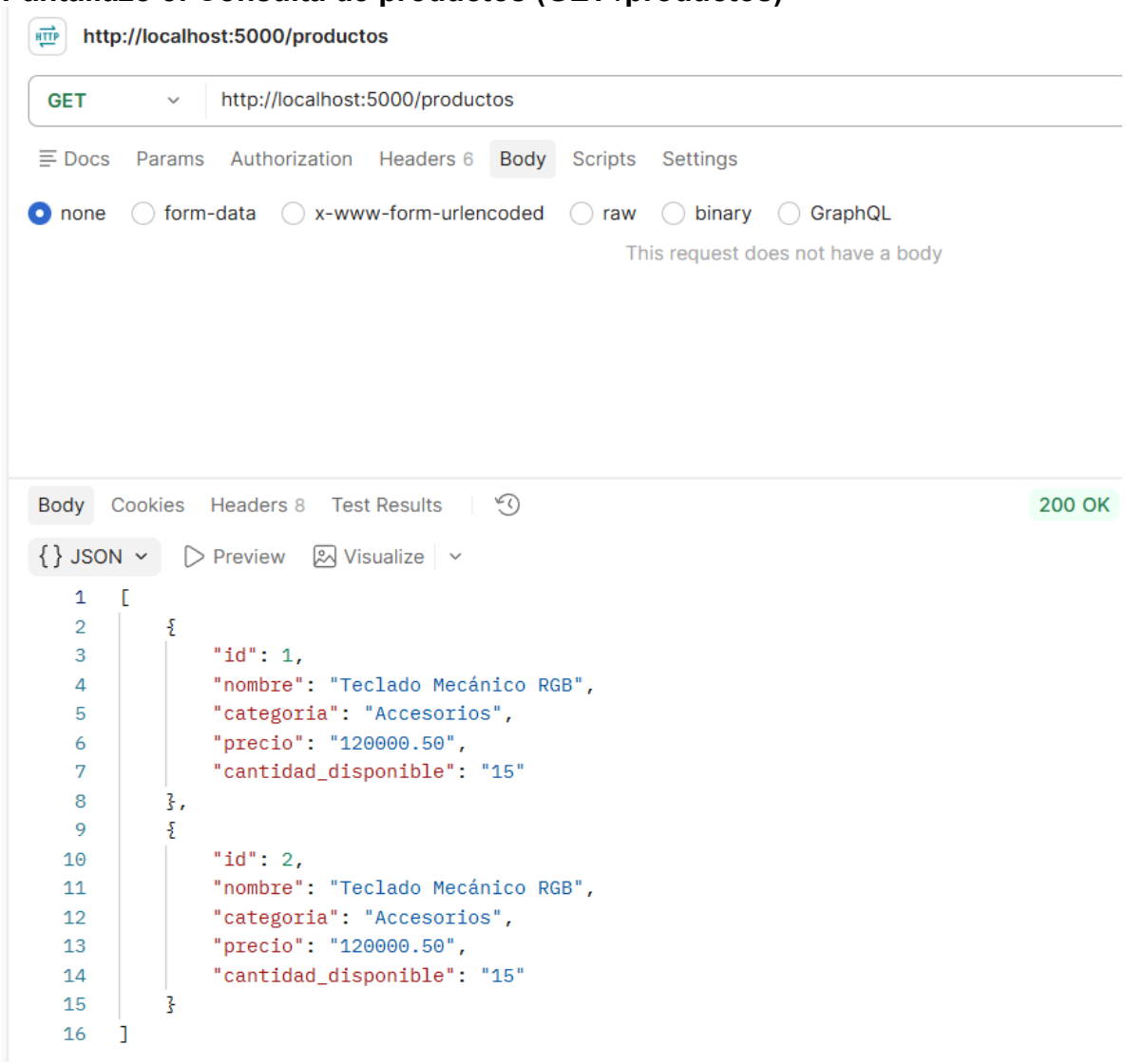
Pantallazo 3: Verificación del servidor (GET /prueba)



Pantallazo 4: Registro de un producto (POST /productos)



Pantallazo 5: Consulta de productos (GET /productos)



PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Fase 1: Investigación y preparación del entorno

1. ¿Qué es Node.js y qué función cumple?

Para mí, Node.js es una herramienta que nos permite ejecutar código JavaScript fuera del navegador. Básicamente, nos sirve para crear y poner en funcionamiento un servidor que se encargue de procesar las peticiones de una aplicación.

2. ¿Qué es Express y por qué se utiliza para desarrollar una API?

Entiendo que Express es una herramienta que funciona con Node.js y nos facilita la creación de servidores y rutas. Se utiliza porque nos ayuda a escribir menos código y a manejar más fácilmente las peticiones y respuestas de nuestra API.

3. ¿Qué es una API REST?

Para mí, una API REST es una forma de permitir que dos aplicaciones se comuniquen entre sí mediante peticiones HTTP. Por ejemplo, podemos utilizar Postman para enviar una petición a nuestro servidor y recibir información en formato JSON.

4. ¿Qué función cumple CORS?

Básicamente, CORS sirve para controlar qué aplicaciones pueden realizar peticiones a nuestro servidor desde un navegador. Esto es importante cuando queremos conectar nuestra API con un frontend desarrollado en herramientas como React o Angular.

5. ¿Qué es MySQL y para qué sirve una base de datos?

Entiendo que MySQL es un sistema que nos permite crear y administrar bases de datos. Estas sirven para guardar información de forma organizada, por ejemplo, los productos de una tienda, y poder consultarla o modificarla cuando sea necesario.

6. ¿Qué información se necesita para establecer una conexión con MySQL en línea?

Para establecer la conexión necesitamos algunos datos importantes, como la dirección del servidor (host), el puerto (port), el usuario (user), la contraseña (password) y el nombre de la base de datos (database). Con esta información, nuestra aplicación puede conectarse al servidor de MySQL.

7. ¿Qué función cumple cada dependencia instalada en el proyecto?

- **Express:** Nos facilita la creación del servidor y las rutas de nuestra API.
- **CORS:** Permite controlar las peticiones que se realizan desde otros orígenes.
- **MySQL2:** Nos permite conectar Node.js con MySQL y ejecutar consultas a la base de datos.
- **Nodemon:** Reinicia automáticamente el servidor cuando hacemos cambios en el código, lo que nos facilita el trabajo mientras desarrollamos.

Fase 4: Revisión y análisis de errores

1. ¿Cuál es el error?

El error consiste en que el nombre de la tabla que utilizamos en la consulta SQL no coincide con el nombre que tiene en la base de datos. Por ejemplo, podemos escribir productos cuando la tabla realmente se llama producto.

2. ¿Por qué ocurre?

Esto ocurre porque podemos equivocarnos al escribir el nombre de la tabla en el código. Aunque parezca un detalle pequeño, la consulta debe apuntar a la tabla correcta para que funcione.

3. ¿Qué consecuencia tiene?

Cuando pasa esto, MySQL no encuentra la tabla que estamos buscando y devuelve un error. Como consecuencia, la API no puede completar correctamente la consulta o el registro de la información.

4. ¿Cómo debería corregirse?

Para solucionarlo, debemos revisar el nombre de la tabla en MySQL y compararlo con el que escribimos en las consultas del código. Después, corregimos el nombre para que coincida con el de la base de datos.

5. ¿Qué prueba permite comprobar la solución?

Podemos probar nuevamente la petición desde Postman, utilizando GET para consultar o POST para registrar información. Si todo funciona correctamente, debemos recibir una respuesta exitosa y comprobar que no aparezca el mismo error en la consola.

Sección 4: Preguntas de análisis

1. ¿Qué diferencia existe entre GET y POST?

Entiendo que GET se utiliza para consultar información que ya existe en el servidor, mientras que POST sirve para enviar información nueva. Por ejemplo, podemos usar GET para listar productos y POST para registrar uno nuevo.

2. ¿Por qué se debe validar la información antes de almacenarla?

Para mí, es importante revisar los datos antes de guardarlos porque así evitamos errores y nos aseguramos de que la información esté completa. Por ejemplo, podemos comprobar que el producto tenga un nombre y que el precio sea correcto antes de registrarlo.

3. ¿Qué sucede cuando la API intenta insertar un registro en una tabla que no existe?

Cuando esto sucede, MySQL devuelve un error porque no encuentra la tabla donde se quiere guardar la información. Por eso, debemos revisar que la tabla exista y que el código esté utilizando el nombre correcto para evitar problemas al registrar los datos.

4. ¿Por qué es importante distinguir entre una respuesta exitosa y una respuesta de error?

Pienso que esto es importante porque nos permite saber si una petición salió bien o si ocurrió algún problema. Por ejemplo, el código 200 indica que la petición se procesó correctamente, mientras que el 500 señala que ocurrió un error interno en el servidor.

5. ¿Qué diferencia existe entre los errores de conexión y los errores de consulta SQL?

La diferencia es que los errores de conexión aparecen cuando nuestra aplicación no logra conectarse con MySQL, por ejemplo, porque los datos de acceso son incorrectos. En cambio, los errores de consulta ocurren cuando sí existe una conexión, pero la consulta SQL tiene algún problema, como un nombre de tabla equivocado.

6. ¿Por qué se deben utilizar consultas parametrizadas?

Entiendo que las consultas parametrizadas nos ayudan a trabajar con los datos de una forma más segura. Se utilizan para reducir el riesgo de ataques de inyección SQL y evitar que la información enviada por el usuario se interprete incorrectamente dentro de una consulta.

7. ¿Qué función cumple CORS cuando la API se consume desde Angular?

Básicamente, CORS permite controlar si una aplicación desarrollada en Angular puede realizar peticiones a nuestra API desde un origen diferente. Esto es necesario cuando el frontend y el backend funcionan en puertos o dominios distintos y el navegador debe comprobar si la comunicación está permitida.

8. ¿Qué ventajas ofrece almacenar la información en una base de datos en línea?

Para mí, una de las principales ventajas es que podemos acceder a la información desde diferentes lugares y dispositivos que tengan conexión a internet. Además, los datos quedan almacenados de forma organizada y pueden ser utilizados por las aplicaciones autorizadas sin depender únicamente de un computador.

9. ¿Por qué es importante probar los casos incorrectos y no solamente los exitosos?

Pienso que no debemos probar solamente cuando todo funciona bien, porque también pueden presentarse errores al ingresar información. Por eso, es importante comprobar cómo responde nuestra API cuando faltan datos o se envía información incorrecta, para encontrar problemas y corregirlos antes de utilizar el sistema.

10. ¿Cómo se puede proteger la información de conexión a la base de datos?

Entiendo que debemos evitar escribir las contraseñas y otros datos privados directamente en el código que compartimos. Una forma de protegerlos es utilizar variables de entorno, por ejemplo, mediante un archivo .env, y evitar subir ese archivo a repositorios públicos como GitHub.